

Introducción a los Contenedores Web

Concepto y Arquitectura

Ing. Alberto Manases Turcios Ortiz
Gestión de Servidores Web – Sección A
Miércoles 15 de julio de 2026



Encuadre de la clase

Unidad 1: Creación de contenedores web para la ejecución de servlets

Horario: 1:00 pm – 2:40 pm

Duración: 1 hora 40 minutos

Objetivos de la clase

1. Explicar con sus propias palabras qué es un contenedor web y por qué existe.
2. Trazar el camino completo de una petición desde el navegador hasta la respuesta.
3. Escribir y ejecutar su primer programa TypeScript que actúa como un mini-contenedor web.



- Mini glosario

- **Cliente:** quien hace la petición (navegador, app, Postman).
- **Servidor:** máquina/programa que escucha y responde.
- **IP:** dirección numérica de una máquina en la red.
- **Puerto:** "puerta" numerada (0-65535) dentro de esa IP.
- **HTTP:** el idioma con el que cliente y servidor se hablan.
- **Terminal:** donde escribimos comandos para correr nuestros programas.
- **Proceso:** un programa en ejecución, con su propia memoria.
- **VM (Máquina Virtual):** una computadora simulada dentro de otra.

Hoy: tomamos esas piezas sueltas y las ensamblamos en un concepto nuevo: el **contenedor web**.



El restaurante



- El **mesero** (servidor web) recibe al cliente, toma su orden y trae el plato. **No cocina.**
- La **cocina** (contenedor web) es donde se prepara la lógica: cálculos, consultas a base de datos.
- El **chef ejecutando una receta** (servlet / route handler) sabe responder a un pedido concreto: GET /ensalada.



¿Cuándo se necesita la cocina?

Solo mesero, sin cocina

Si el restaurante **solo sirve agua embotellada** que ya viene lista, eso es un **servidor web puro**: sirve contenido **estático** (HTML, imágenes) que ya existe, sin "cocinar" nada.

Aparece la cocina

En cuanto alguien pide algo que **hay que preparar a la medida** – un total con impuestos, un saludo personalizado, datos desde una base de datos – **se necesita la cocina**: el contenedor.

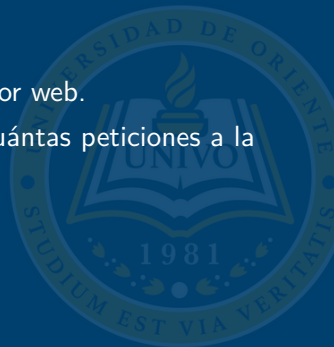
Regla práctica: si la respuesta cambia según quién pregunta o qué pregunta, necesitas contenedor.



¿Qué es, formalmente, un contenedor web?

Un **contenedor web** es un programa que:

1. **Recibe** la petición HTTP que el servidor web le delega.
2. **Decide** qué pieza de código (*servlet*) debe atenderla.
3. **Ejecuta** esa pieza de código y devuelve el resultado al servidor web.
4. **Administra** sesiones de usuario, seguridad y concurrencia (cuántas peticiones a la vez).





El recorrido completo de una petición

1. Usuario escribe `http://miapp.com/saludo?nombre=Ana`



2. El **navegador (CLIENTE)** envía la petición HTTP



3. El **SERVIDOR WEB** la recibe en el puerto correspondiente



4. `/saludo` necesita lógica → se delega al **CONTENEDOR**



5. El contenedor busca el **route handler** de `/saludo`



6. El handler lee `nombre` y construye la respuesta



7. Respuesta: CONTENEDOR → SERVIDOR WEB → navegador

Resultado: el usuario ve "Hola, Ana"



El ciclo de vida – el turno de un empleado



Momento	Qué pasa	Analogía
init	Carga el código una sola vez	Empleado llega y se uniforma
service	Se ejecuta por cada petición	Atiende cada cliente, uno a uno
destroy	Libera todo al apagar la app	Cierra su estación al final



Comparando contenedores reales de la industria

Contenedor	Lenguaje	Licencia	Uso típico
PHP-FPM + Nginx	PHP	Libre (PHP License)	Sitios y APIs PHP (WordPress, Laravel)
Node.js + Express	TypeScript/JavaScript	Libre (MIT)	APIs y apps web modernas (este curso)
Microsoft IIS	.NET / ASP	Propietaria (Windows)	Apps corporativas .NET
Gunicorn + Django	Python	Libre (BSD)	APIs y apps Python empresariales



Antes de programar: preparar el proyecto

```
1 mkdir mini-contenedor && cd mini-contenedor
2 npm init -y
3 npm install --save-dev typescript @types/node
4 npx tsc --init
```

- `npm init -y`: crea `package.json` (la "ficha" del proyecto).
- `typescript` y `@types/node`: nos dan tipado y autocompletado para el módulo `http` de Node.
- `tsc --init`: genera `tsconfig.json`, la configuración del compilador.



Nuestro primer "contenedor" (1/2): imports y handler

```
1 import * as http from "http";
2 import { URL } from "url";
3
4 // route handler -- equivalente a un servlet
5 function manejarSaludo(nombre: string): string {
6     return `

# Hola, ${nombre}!</h1> 7 <p>Bienvenido a tu primer contenedor web.</p>`; 8 }


```

El handler es solo una función: recibe un dato, devuelve un texto. Nada "mágico" todavía.



Nuestro primer "contenedor" (2/2): el servidor HTTP

```
1 // el "contenedor": decide QUE handler ejecutar segun la ruta
2 const servidor = http.createServer((req, res) => {
3   const url = new URL(req.url ?? "/", `http://${req.headers.host}`);
4
5   if (url.pathname === "/saludo") {
6     const nombre = url.searchParams.get("nombre") ?? "desconocido";
7     res.writeHead(200, { "Content-Type": "text/html; charset=utf-8" });
8     res.end(manejarSaludo(nombre));
9     return;
10  }
11  res.writeHead(404, { "Content-Type": "text/plain" });
12  res.end("Ruta no encontrada");
13 });
14
15 servidor.listen(3000, () => console.log("Escuchando en :3000"));
```



Línea por línea: ¿qué hace cada parte?

- `http.createServer(...)` crea el **contenedor**: un proceso que escucha peticiones.
- El callback `(req, res)` recibe la petición (`req`) y el objeto para responder (`res`).
- `new URL(...)` descompone la URL para leer `pathname` y `searchParams` (los `?clave=valor`).
- El `if (url.pathname === /saludo")` es el **corazón del contenedor**: decide qué handler corre.
- `manejarSaludo(...)` es el **servlet / route handler**: la unidad de lógica que responde.
- Si ninguna ruta coincide → 404: "no sé atender ese pedido".



Ejecutarlo y probarlo

```
1 npm install --save-dev ts-node    # si no lo tienen aun
2 npx ts-node servidor.ts
```

Abrir el navegador en:
<http://localhost:3000/saludo?nombre=Ana>

Deben ver: Hola, Ana!

Idea clave

Acaban de construir, en miniatura, exactamente lo que hace un contenedor real (como IIS) por dentro: recibir, decidir, responder. El resto del curso son versiones más completas de este mismo patrón.



Ejemplo adicional resuelto: /suma

Antes del ejercicio individual, veamos juntos **otra ruta completa** para fijar el patrón:

```
1 if (url.pathname === "/suma") {  
2   const a = Number(url.searchParams.get("a") ?? 0);  
3   const b = Number(url.searchParams.get("b") ?? 0);  
4   res.writeHead(200, { "Content-Type": "text/plain; charset=utf-8"  
     });  
5   res.end(`El resultado de ${a} + ${b} es ${a + b}`);  
6   return;  
7 }
```

Probar en: <http://localhost:3000/suma?a=4&b=5> → El resultado de 4 + 5
es 9

*Nota: cada **if** nuevo es una ruta nueva que el "contenedor" sabe atender. En clases futuras, Express nos va a evitar escribir tantos **if**.*



Ejercicio individual

1. Ejecutar el código de `servidor.ts` y comprobar que funciona con al menos 2 nombres distintos.
2. Agregar una ruta `/despedida` que reciba nombre y responda:
`.^diós, [nombre]. ¡Vuelve pronto!"`
3. Probar qué pasa al visitar una ruta que no existe (ej. `/foo`). ¿Qué mensaje ven?
¿Por qué?

Pista

Fíjense en cómo está armado el bloque de `/saludo` y el de `/suma` que acabamos de ver juntos. `/despedida` sigue **exactamente el mismo molde**: leer parámetro → construir texto → `res.end(...)`.



Verificación de aprendizaje

1. En la analogía del restaurante, ¿qué representa la cocina?





Verificación de aprendizaje

1. En la analogía del restaurante, ¿qué representa la cocina?

El contenedor web.





Verificación de aprendizaje

1. En la analogía del restaurante, ¿qué representa la cocina?
El contenedor web.
2. ¿Cuáles son los 3 momentos del ciclo de vida de un contenedor?





Verificación de aprendizaje

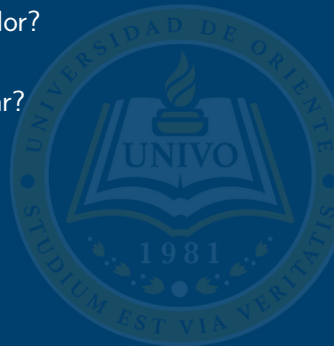
1. En la analogía del restaurante, ¿qué representa la cocina?
El contenedor web.
2. ¿Cuáles son los 3 momentos del ciclo de vida de un contenedor?
Inicio (`init`), atención por petición, cierre (`destroy`).





Verificación de aprendizaje

1. En la analogía del restaurante, ¿qué representa la cocina?
El contenedor web.
2. ¿Cuáles son los 3 momentos del ciclo de vida de un contenedor?
Inicio (`init`), atención por petición, cierre (`destroy`).
3. En nuestro TypeScript, ¿qué línea decide qué handler ejecutar?





Verificación de aprendizaje

1. En la analogía del restaurante, ¿qué representa la cocina?

El contenedor web.

2. ¿Cuáles son los 3 momentos del ciclo de vida de un contenedor?

Inicio (`init`), atención por petición, cierre (`destroy`).

3. En nuestro TypeScript, ¿qué línea decide qué handler ejecutar?

El `if (url.pathname === /saludo")` dentro de `http.createServer`.





Verificación de aprendizaje

1. En la analogía del restaurante, ¿qué representa la cocina?
El contenedor web.
2. ¿Cuáles son los 3 momentos del ciclo de vida de un contenedor?
Inicio (`init`), atención por petición, cierre (`destroy`).
3. En nuestro TypeScript, ¿qué línea decide qué handler ejecutar?
El `if (url.pathname === /saludo")` dentro de `http.createServer`.
4. ¿Cómo se llama, en la terminología oficial de la industria, la pieza de código que atiende una ruta?



Verificación de aprendizaje

1. En la analogía del restaurante, ¿qué representa la cocina?
El `contenedor web`.
2. ¿Cuáles son los 3 momentos del ciclo de vida de un contenedor?
Inicio (`init`), atención por petición, cierre (`destroy`).
3. En nuestro TypeScript, ¿qué línea decide qué handler ejecutar?
El `if (url.pathname === /saludo")` dentro de `http.createServer`.
4. ¿Cómo se llama, en la terminología oficial de la industria, la pieza de código que atiende una ruta?
`Servlet`.



Resumen de la clase

Concepto	Idea clave
Contenedor web	Recibe, decide y ejecuta lógica dinámica
Servlet / route handler	Unidad de código que atiende una ruta específica
Ciclo de vida	Inicio → atención por petición → cierre
<code>http.createServer</code>	La forma más básica de construir un contenedor en TypeScript



Tarea para casa

1. Completar el ejercicio de la ruta /despedida.
2. Investigar y comentar el Foro 1: nombres de **3 contenedores/servidores de aplicaciones** usados en la industria (uno debe ser propietario), y una empresa real que use cada uno.

Para profundizar (documentación oficial)

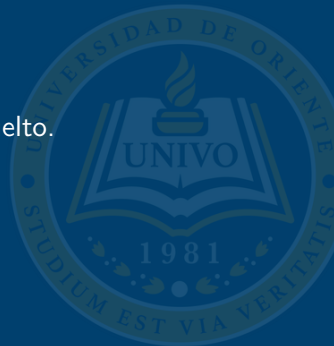
- nodejs.org – módulo `http`
- expressjs.com – el framework que simplificará esto en próximas clases



Preparación para la Clase 3

Tema: Diferencias entre contenedor web y servidor web – profundización con un segundo ejemplo en TypeScript.

- Repasar bien la analogía del restaurante: la van a necesitar.
- Traer su `servidor.ts` funcionando con `/despedida` ya resuelto.





Comisión de Acreditación
de la Calidad Académica
UNIVERSIDAD DE ORIENTE - UNIVO
ACREDITADA
2022 - 2027

MUCHAS GRACIAS

Te espero la próxima clase

